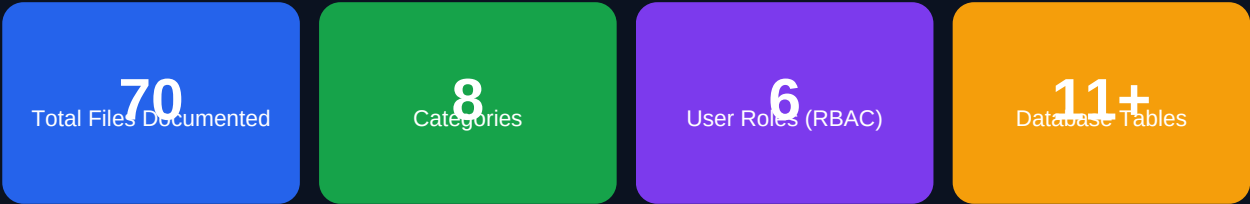


COMPLETE FILE-BY-FILE GUIDE

Dental CRM Project

Har file ka maqsad, explanation, aur role — ek hi jagah, taake presentation ke waqt aap kisi bhi file ke baray mein confident ho kar bata sakein.



Frontend stack	Next.js 16, React 19, TypeScript, Tailwind CSS v4, Drizzle ORM
Backend stack	FastAPI, SQLAlchemy, Pydantic, PyJWT, Bcrypt, OpenAI, pgvector
Architecture	3-Tier Clean Architecture (API → Service → Model/ORM)

Table of Contents

Architecture at a Glance

Presentation ke liye ek line

Technology Stack Summary

RBAC — Roles & Permissions

File Count Summary

Quick File Index (A–Z)

3

3

4

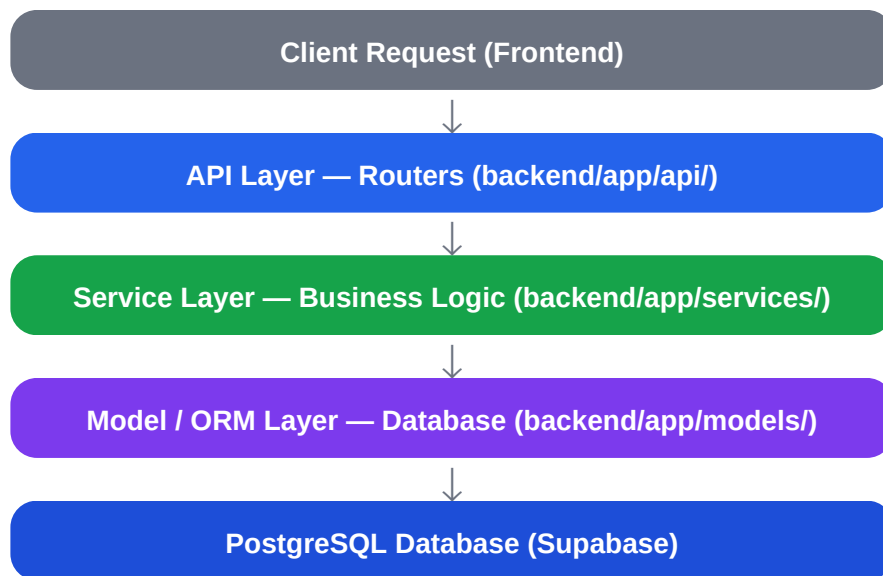
4

13

14

Architecture at a Glance

Is project ka backend **3-Tier Clean Architecture** follow karta hai. Har request is flow se guzarti hai:



Pydantic Schemas is poore flow ke sath-sath kaam karte hain — har incoming request aur outgoing response ko validate karte hain, taake sirf sahi-shaped data hi system ke andar/bahar jaye.

Presentation ke liye ek line

“Hamara backend **3-Tier Clean Architecture** follow karta hai: **API Layer (Routers)** request leti hai, **Service Layer** business logic chalati hai, aur **Model / ORM Layer** database se safe query karti hai, jabke **Pydantic** input/output ko validate karta hai.”

Technology Stack Summary

Layer	Technology
Frontend framework	Next.js 16 + React 19
Frontend language	TypeScript
Styling	Tailwind CSS v4
Frontend ORM	Drizzle ORM
Backend framework	FastAPI (Uvicorn server)
Backend ORM	SQLAlchemy
Validation	Pydantic (BaseSettings + schemas)
Auth & Security	PyJWT, Bcrypt
Database & Auth Platform	Supabase (PostgreSQL)
AI	OpenAI API + pgvector (RAG embeddings)
Migrations	Alembic
API Docs	OpenAPI (/docs) + Scalar (/scalar)

RBAC — Roles & Permissions

backend/app/core/roles.py mein defined 6 user roles, granular permission mapping ke sath:

super_admin	Poore platform ka top-level access — sab organizations manage kar sakta hai.
org_admin	Apni organization (clinic brand) ke andar full control.
clinic_manager	Ek specific clinic branch ka operational control.
agent	Leads, calls, aur follow-ups handle karne wala staff.
reception	Appointments, check-in, aur front-desk operations.
finance	Revenue, invoices, aur payments manage karta hai.

In 6 roles par 50+ granular permissions map hoti hain — matlab har role sirf wahi actions kar sakta hai jo uske liye explicitly allow ki gayi hain (e.g. finance role revenue dekh/manage kar sakta hai, lekin clinic settings nahi badal sakta).

1. Root Configuration & Utility Files

Project ki jaan-pehchaan: dependencies, config, aur utility scripts

`package.json`

Frontend Dependencies: Next.js 16, React 19, TypeScript, Tailwind CSS v4, aur Drizzle ORM ke packages aur scripts (`npm run dev`, `npm run build`) define karti hai.

`tsconfig.json`

TypeScript Config: Next.js aur TypeScript compiler ke rules (strict mode, path aliases `@/*`) define karta hai.

`next.config.ts`

Next.js Engine Config: Next.js framework ki build aur runtime configurations set karta hai.

`drizzle.config.json`

Drizzle ORM Config: PostgreSQL database connection string aur schema files ka path define karta hai.

`postcss.config.mjs` &
`eslint.config.mjs`

Styling & Linter: Tailwind CSS processing aur code quality/formatting ke rules set karti hain.

`requirements.txt`

Python Backend Dependencies: FastAPI, Uvicorn, SQLAlchemy, Pydantic, Supabase, PyJWT, Bcrypt, OpenAI, pgvector wagara ke packages list karti hai.

`alembic.ini`

Alembic Config: Database schema migrations chalane ke liye configuration file hai.

`generate_hash.py`

Password Hash Utility: CLI se direct bcrypt password hash generate karne ki helper script.

`generate_project_pdf.py`

PDF Report Generator: ReportLab library se project summary ya invoices ki PDF create karne ki script.

`test_dashboard.html`

Standalone HTML Test Page: Browser se directly backend APIs (Login, Leads, Appointments) test karne ke liye quick testing UI dashboard.

`PROJECT_SUMMARY.md`

Project Architecture Summary: Poore project ki overview, user roles aur permission matrix ka quick reference.

11 files in this category.

2. Frontend Application (src/)

Next.js app ka bunyadi structure

`src/app/layout.tsx`

Root HTML Layout: Poore web app ka master shell/wrapper jisme metadata, global fonts, aur body tags define hain.

`src/app/page.tsx`

Main Home Page: Web application ka main landing page (React Server Component).

`src/app/globals.css`

Global CSS: Tailwind CSS rules aur project ke global styling variables.

`src/app/api/health/route.ts`

Next.js Health Route: Next.js server ki live status check karne ka API endpoint.

`src/db/index.ts`

Drizzle Client Init: Node.js pg pool driver ke zariye PostgreSQL database connection banata hai.

`src/db/schema.ts`

Drizzle Schemas: TypeScript side par database tables aur types define karta hai.

6 files in this category.

3. Backend Core Engine

FastAPI ka dil — entry point, DB connection, security

`backend/app/main.py`

Application Entry Point: FastAPI app create karta hai, CORS middleware configure karta hai, lifespan (startup/shutdown events) handle karta hai, aur saare 10 API routers attach karta hai. OpenAPI docs ([/docs](#)) aur Scalar docs ([/scalar](#)) yahin se serve hoti hain.

`backend/app/database.py`

Database Connection: SQLAlchemy engine, Session maker ([SessionLocal](#)), aur [get_db\(\)](#) dependency generator banata hai jo har request par DB session deta hai aur close karta hai.

`backend/app/core/config.py`

Environment Settings: [.env](#) file se environment variables (Database URL, JWT Secret Key, Supabase Keys, Project Name) Pydantic [BaseSettings](#) me load karta hai.

`backend/app/core/roles.py`

RBAC Engine: 6 User Roles ([super_admin](#), [org_admin](#), [clinic_manager](#), [agent](#), [reception](#), [finance](#)) aur 50+ Permissions ka granular mapping matrix store karta hai.

`backend/app/core/auth_utils.py`

Security Hashing: [hash_password\(\)](#) aur [verify_password\(\)](#) functions jo passwords ko safe tareeqay se Bcrypt salt ke sath hash aur verify karte hain.

`backend/app/core/supabase_client.py`

Supabase SDK Connection: Anon Key aur Service Role Key ke zariye Supabase cloud clients initialize karta hai.

6 files in this category.

4. API Endpoints / Routers (backend/app/api/)

Frontend se aane wali requests yahan receive hoti hain

Yeh files frontend se aane wali HTTP requests ko receive karti hain, authorization check karti hain aur services ko call karti hain:

`backend/app/api/auth.py`

Authentication: User Registration (`/register`), User Login (`/login`), Profile Info (`/me`), aur Logout endpoints.

`backend/app/api/users.py`

User Management: Staff users ko create, update, delete, role assign, aur list karne ke endpoints.

`backend/app/api/organizations.py`

Organization CRUD: Dental clinic brands / multi-tenant organizations create aur manage karne ke routes.

`backend/app/api/clinics.py`

Clinic Branches: Specific branch/clinic create karne, unke timings aur status manage karne ke routes.

`backend/app/api/leads.py`

Leads & Inquiries: New patient leads capture karna, agents ko assign karna, status badalna, calls/notes log karna.

`backend/app/api/appointments.py`

Appointment Booking: Doctor appointments book karna, reschedule karna, cancel karna, aur reception check-in handle karna.

`backend/app/api/revenue.py`

Financial Operations: Invoices generate karna, payments receive karna, installments aur refunds manage karna.

`backend/app/api/reports.py`

Analytics & Dashboards: Role-specific dashboards, agent performance, clinic revenue, aur conversion rate reports calculate karna.

`backend/app/api/audit.py`

System Audit Logs: Security audit logs view aur filter karna taake compliance maintain rahay.

`backend/app/api/ai.py`

AI & Governance: AI lead summarization (`/summarize-lead`), Prompt versioning, AI usage/token tracking, automation rules, aur vector knowledge base routes.

10 files in this category.

5. Database Models (backend/app/models/)

SQLAlchemy ORM classes — PostgreSQL tables ka structure

Yeh SQLAlchemy ORM classes hain jo PostgreSQL database tables ke structure, columns, types, foreign keys aur relationships ko define karti hain:

`organization.py`

organizations table: Multi-tenant parent organization ka record (name, domain, logo, status).

`clinic.py`

clinics table: Physical clinic branch (address, phone, timezone, active status).

`user.py`

users table: Staff members, unke hashed passwords, role, organization_id, clinic_id.

`lead.py`

leads table: Patient inquiry details, stage, status, assigned agent, treatment interest.

`appointment.py`

appointments table: Doctor appointment timing, duration, status (scheduled, completed, checkin, noshow).

`revenue.py`

revenues & payments tables: Invoices, payment mode (cash, card, online), installments, and refunds.

`call.py`

calls table: Agent ki patient ke sath phone call logs (duration, outcome, recording url).

`note.py`

notes table: Patient timeline par agents aur doctors ke text notes.

`task.py`

tasks table: Follow-up tasks jo kisi agent ya receptionist ko assign hotay hain with due date.

`audit.py`

audit_logs table: Har record par hone wali tabdeeli ka snapshot (old values, new values, user, IP).

`ai_automation.py`

ai_prompt_versions, **ai_runs**, **ai_feedback**, **ai_usage**, **automation_rules**, **knowledge_documents**, aur **knowledge_chunks** (1536-dimensional pgvector embeddings table for RAG).

11 files in this category.

6. Pydantic Schemas (backend/app/schemas/)

Incoming/outgoing JSON data validation

Yeh files incoming JSON requests aur outgoing JSON responses ka data validate karti hain (e.g. Email format theek hai? Required fields mojood hain?):

`user.py`

`UserCreate`, `UserUpdate`, `UserLogin`, `Token`, `UserResponse` schemas.

`organization.py`

`OrganizationCreate`, `OrganizationUpdate`, `OrganizationResponse`.

`clinic.py`

`ClinicCreate`, `ClinicUpdate`, `ClinicResponse`.

`lead.py`

`LeadCreate`, `LeadUpdate`, `LeadStatusUpdate`, `LeadResponse`.

`appointment.py`

`AppointmentCreate`, `AppointmentReschedule`, `AppointmentCheckin`.

`revenue.py`

`RevenueCreate`, `PaymentCreate`, `RefundRequest`.

`call.py`

`CallLogCreate`, `CallResponse`.

`note.py`

`NoteCreate`, `NoteResponse`.

`task.py`

`TaskCreate`, `TaskUpdate`, `TaskStatusUpdate`.

`report.py`

Analytics and dashboard response data structures.

`audit.py`

`AuditLogResponse` and audit filtering criteria schemas.

`ai_automation.py`

AI run logs, prompt version creation, token consumption, vector chunk creation schemas.

12 files in this category.

7. Business Logic Services (backend/app/services/)

Controller aur Database ke darmiyan asal logic

Controller/API aur Database ke darmian asal business logic yahan likhi gayi hai:

backend/app/services/auth.py

User credentials check karta hai, JWT token generate karta hai, aur current user ko verify karta hai.

backend/app/services/user.py

User creation, role update, user list filtering based on organization.

backend/app/services/organization.py

Organization lifecycle, limits aur multi-tenant policies implement karta hai.

backend/app/services/clinic.py

Clinic creation, clinic manager assignment, clinic statistics.

backend/app/services/lead.py

Lead auto-assignment rules, status transitions, call & note attachments.

backend/app/services/appointment.py

Calendar collision checks (do appointments aik time pe clash na hon), check-in calculations.

backend/app/services/revenue.py

Invoice balance calculations, partial payments, refund processing, commission calculations.

backend/app/services/report.py

Aggregation queries: Total sales, lead conversion rates, doctor productivity metrics.

backend/app/services/audit.py

Database me har critical event ka log create karna.

backend/app/services/openai_service.py

OpenAI API ko call karke lead conversation ki AI summary generate karta hai.

backend/app/services/ai_automation.py

Prompt versions manage karna, AI latency/token cost save karna, vector chunks create karna.

11 files in this category.

8. Database Migrations (backend/migrations/ & alembic/)

Schema evolution aur version control

`backend/migrations/001_initial_schema.sql`

Supabase Master SQL: 11 PostgreSQL tables, 15 enum types, 30+ indexes, triggers, aur Row Level Security (RLS) policies create karne ki master SQL script.

`alembic/env.py`

Alembic Python environment jo SQLAlchemy models aur DB connection ko connect karta hai.

`alembic/versions/`

Incremental migration scripts (Baseline schema, RLS policies, AI automation tables, password columns).

3 files in this category.

File Count Summary

Quick statistics — kis category mein kitni files hain:

Category	File Count
1. Root Configuration & Utility Files	11
2. Frontend Application (src/)	6
3. Backend Core Engine	6
4. API Endpoints / Routers (backend/app/api/)	10
5. Database Models (backend/app/models/)	11
6. Pydantic Schemas (backend/app/schemas/)	12
7. Business Logic Services (backend/app/services/)	11
8. Database Migrations (backend/migrations/ & alembic/)	3
Total	70

Quick File Index (A–Z)

Presentation ke doraan agar koi specific file ka naam pooche, to isse jaldi dhoond sakte hain — sab files alphabetically, unki category ke saath:

File	Category
alembic/versions/	8. Database Migrations (backend/migrations/ & alembic/)
backend/migrations/001_initial_schema.sql	8. Database Migrations (backend/migrations/ & alembic/)
backend/app/api/ai.py	4. API Endpoints / Routers (backend/app/api/)
ai_automation.py	5. Database Models (backend/app/models/)
ai_automation.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/ai_automation.py	7. Business Logic Services (backend/app/services/)
alembic.ini	1. Root Configuration & Utility Files
appointment.py	5. Database Models (backend/app/models/)
appointment.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/appointment.py	7. Business Logic Services (backend/app/services/)
backend/app/api/appointments.py	4. API Endpoints / Routers (backend/app/api/)
backend/app/api/audit.py	4. API Endpoints / Routers (backend/app/api/)
audit.py	5. Database Models (backend/app/models/)
audit.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/audit.py	7. Business Logic Services (backend/app/services/)
backend/app/api/auth.py	4. API Endpoints / Routers (backend/app/api/)
backend/app/services/auth.py	7. Business Logic Services (backend/app/services/)
backend/app/core/auth_utils.py	3. Backend Core Engine
call.py	5. Database Models (backend/app/models/)
call.py	6. Pydantic Schemas (backend/app/schemas/)
clinic.py	5. Database Models (backend/app/models/)
clinic.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/clinic.py	7. Business Logic Services (backend/app/services/)

File	Category
backend/app/api/clinics.py	4. API Endpoints / Routers (backend/app/api/)
backend/app/core/config.py	3. Backend Core Engine
backend/app/database.py	3. Backend Core Engine
drizzle.config.json	1. Root Configuration & Utility Files
alembic/env.py	8. Database Migrations (backend/migrations/ & alembic/)
generate_hash.py	1. Root Configuration & Utility Files
generate_project_pdf.py	1. Root Configuration & Utility Files
src/app/globals.css	2. Frontend Application (src/)
src/db/index.ts	2. Frontend Application (src/)
src/app/layout.tsx	2. Frontend Application (src/)
lead.py	5. Database Models (backend/app/models/)
lead.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/lead.py	7. Business Logic Services (backend/app/services/)
backend/app/api/leads.py	4. API Endpoints / Routers (backend/app/api/)
backend/app/main.py	3. Backend Core Engine
next.config.ts	1. Root Configuration & Utility Files
note.py	5. Database Models (backend/app/models/)
note.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/openai_service.py	7. Business Logic Services (backend/app/services/)
organization.py	5. Database Models (backend/app/models/)
organization.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/organization.py	7. Business Logic Services (backend/app/services/)
backend/app/api/organizations.py	4. API Endpoints / Routers (backend/app/api/)
package.json	1. Root Configuration & Utility Files
src/app/page.tsx	2. Frontend Application (src/)
postcss.config.mjs & eslint.config.mjs	1. Root Configuration & Utility Files
PROJECT_SUMMARY.md	1. Root Configuration & Utility Files

File	Category
report.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/report.py	7. Business Logic Services (backend/app/services/)
backend/app/api/reports.py	4. API Endpoints / Routers (backend/app/api/)
requirements.txt	1. Root Configuration & Utility Files
backend/app/api/revenue.py	4. API Endpoints / Routers (backend/app/api/)
revenue.py	5. Database Models (backend/app/models/)
revenue.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/revenue.py	7. Business Logic Services (backend/app/services/)
backend/app/core/roles.py	3. Backend Core Engine
src/app/api/health/route.ts	2. Frontend Application (src/)
src/db/schema.ts	2. Frontend Application (src/)
backend/app/core/supabase_client.py	3. Backend Core Engine
task.py	5. Database Models (backend/app/models/)
task.py	6. Pydantic Schemas (backend/app/schemas/)
test_dashboard.html	1. Root Configuration & Utility Files
tsconfig.json	1. Root Configuration & Utility Files
user.py	5. Database Models (backend/app/models/)
user.py	6. Pydantic Schemas (backend/app/schemas/)
backend/app/services/user.py	7. Business Logic Services (backend/app/services/)
backend/app/api/users.py	4. API Endpoints / Routers (backend/app/api/)